

Test Summary Report

Campus Equipment Booking & Return System

Course: Software Testing and Validation - Addis Ababa University **Group members:** *(add names and student IDs here)*

1. Executive summary

This report covers testing an existing application: an audit found 4 real defects (one High severity, affecting the core approval workflow) and one required feature missing entirely (cancellation). All were fixed. 115 of 127 automated tests have actually been executed and pass; the remaining 12 (Selenium E2E) are written correctly against a full Page Object Model but could not be run against a real browser in the development sandbox used to build this - see section 7.

2. Product tested

A Flask application for booking shared campus equipment: two roles (student/admin), an equipment catalogue with maintenance status, and a booking lifecycle governed by an explicit state machine.

3. Testing performed

A manual defect audit (live reproduction via curl before writing any tests), unit tests (business logic, one genuine `unittest.mock` test double plus clock-injection throughout), integration tests (Flask routes + real DB + auth together - this is what caught DEF-002), and a Selenium E2E suite with a full Page Object Model.

4. Test execution summary

See docs/metrics.md section 1. Headline: 115/115 executed tests pass (100%); 12 E2E tests are written but blocked from execution here.

5. Unit-test results

88/88 pass. 99% branch coverage on `booking_service.py`, 100% on `auth.py`. Includes both a clock-injection seam (`now=` parameters, used throughout) and one fully isolated `unittest.mock`-based test (`test_booking_service_isolated.py`) that never touches a database.

6. Integration-test results

27/27 pass. Covers registration/auth/authorization together (including a direct regression test for the DEF-001 admin-nav bug), equipment CRUD, and the complete booking workflow including the newly-added cancellation routes. This is the level at which DEF-002 was actually discovered.

7. E2E/Selenium results

12/12 journeys written, covering every required scenario. **Not executed against a real browser.** Chromium/Firefox on the Ubuntu 24.04 image used to build this are snap-only packages with no snap-store network access in that sandbox. A real, standalone Chromium binary (from the Sparticuz project's GitHub releases, intended for AWS Lambda) was located and partially tested - it runs and reports its version correctly, but its GPU/renderer process could not be made to complete a headless page render in that specific container environment even after supplying the missing swiftshader/font libraries, and the effort to work around a Lambda-specific sandboxing quirk was not a good use of further time versus simply running the suite in GitHub Actions, where a full, standard Chrome install is provisioned by `browser-actions/setup-chrome` and this is a solved problem. **Action required before submission:** push to GitHub, let the e2e job in `.github/workflows/ci.yml` run for real, and update this section with the actual result.

8. Formal testing results

All EP, BVA, decision-table, and state-transition cases in the Test Design Document map to real, passing

tests. BVA-3 (the “request exactly the remaining capacity” boundary) is worth calling out specifically: it is the exact case DEF-002 got wrong before the fix, which is about as direct a demonstration as possible that boundary-value analysis targets real risk, not just textbook completeness.

9. Requirements coverage

Every requirement in the Test Design Document’s requirements table has at least one passing automated test (see that document’s per-technique mapping tables).

10. Automation coverage

100% of written test cases are automated.

11. Defects discovered

4, all closed (see docs/defect_log.md): a UI/template inconsistency hiding admin navigation (Medium), a High-severity approval-logic bug that would fail in the common case of booking an item at full capacity, an orphaned/conflicting dead code module (Low), and a non-functional form field (Low). One required feature (booking cancellation) was found entirely missing and was implemented.

12. Defects resolved

4/4 closed, fixed, and covered by regression tests. 0 open.

13. Remaining risks

- **E2E execution gap** (section 7): confirm in a real CI run before relying on the “complete browser journey” claim.
- **Jenkins not yet executed**: the Jenkinsfile mirrors the GitHub Actions stages but has not been run (Docker-in-Docker unavailable in the sandbox this was built in).
- **routes.py coverage (78%)** is lower than the service layer; most gaps are error-flash branches better suited to E2E verification than more integration tests, but worth a second look once E2E is confirmed running.

14. Quality metrics

See docs/metrics.md.

15. CI results

.github/workflows/ci.yml is written and staged (unit+integration, then E2E gated on that job passing) but has not yet been run against a real GitHub repository from this environment. Push and confirm before submission.

16. Lessons learned

- Writing a full lifecycle integration test (request -> approve -> activate -> return) rather than testing each action in isolation is what surfaced DEF-002; testing each admin action against a freshly-seeded REQUESTED booking in isolation (which is what the original codebase’s own hypothetical tests would likely have done) would not have hit the self-counting bug, since the bug only manifests when the booking’s own row already exists in the reserved-quantity query at approval time.
- A single source of truth for “is this user an admin” (is_admin as a property) is worth adding even to a small app - two templates had silently drifted onto two different ways of asking the same question.
- When a real browser genuinely isn’t available, the right move is to try a real alternative, document the attempt and why it didn’t pan out, and move on - not to skip the E2E layer silently or fabricate a result.

17. Limitations

See sections 7, 13, 15.

18. Final quality assessment

The core business logic is now correct (a real, serious workflow-breaking bug was found and fixed) and thoroughly tested. The application layer is integration-tested but not yet confirmed via a real browser run.

19. Recommendations

Not yet ready for final submission as-is. Before submitting: (1) push to GitHub and get a real green Actions run including the E2E job; (2) run the Jenkins pipeline locally and capture output; (3) update sections 7, 13, 15 with the real results; (4) add group member names/IDs to all four documents.

20. Conclusion

Testing this existing codebase found and fixed a defect serious enough that the application's core workflow would have failed in ordinary use - evidence that the testing effort here did real work rather than just producing documentation around already-correct code.

21. Rubric/requirement coverage

See the Test Design Document and Test Plan for the full concept-to-test mapping.

Foundations reflection

Take DEF-002 - `approve_booking()` failing whenever a booking's requested quantity equals the equipment's entire remaining capacity.

Error, fault, failure: the *error* was a design omission made when `calculate_reserved_quantity` was first written: nothing in its signature allowed a caller to say "exclude this one booking from the count," because the author was likely thinking about the *request-time* check (where the new booking doesn't exist in the database yet, so there's nothing to exclude) and didn't separately consider the *approval-time* re-check (where the booking already exists and is REQUESTED, so it does need excluding). That omission produced a *fault*: `get_overlapping_bookings` counted a booking against itself. The fault only becomes a *failure* when someone actually tries to approve a booking that uses all the remaining stock - for equipment with plenty of spare capacity, the fault sits latent and every approval appears to work fine, which is exactly why casual manual testing (which tends to use "obviously fine" test data) would likely have missed it.

Verification or validation? This was caught by **verification** -

`test_it305_full_lifecycle_approve_activate_return` checks the application's actual behaviour against its own specification (a booking that should be approvable, is approved), independent of any end user's judgment about whether that's the *right* specification. It is "are we building the product right," not "are we building the right product." Had this shipped without that integration test, it would eventually have been caught by validation instead - a student and an admin trying to book and approve the department's only projector, and the admin reporting that approval mysteriously fails - but that route to detection is strictly worse: it costs a real user's time and trust, and it happens after the code is already in front of them rather than before.